U7280249 TECHNICAL REPORT

Assignment 3 - Lab Group 1600-1800 Thursdays

Oliver Walters
Tutor: Elise Palethorpe & Donovan Crichton

Introduction

The Haskell modules developed contain the following AIs:

AI:	Description:
`firstLegalMove`	Performs the first legal move
`greedy`	Takes the game state with the best heuristic
`Eliminator`	Eliminates certain parts of the tree, assuming a greedy opponent (only works as player 2)
`oldabpai`	Uses Alpha-Beta Pruning with a simple score heuristic
`default/abpai`	Uses Alpha-Beta Pruning with a weighted-board heuristic

The Haskell modules also contain the following heuristics:

Heuristic:	Description:
`simple`	The value is equivalent to the current score
`weighted`	The value is determined by individual pieces

Warning: The code is developed with the `Int` type class being bounded by the 64-bit integer limit. If the code is run on a 32-bit system, modifications to the definition of `inf` in `src/AI.hs` may need to be made (in-built unit tests check for the upper bound).

Note: All code contributing to the Eliminator function was not tested and was abandoned to develop more complex algorithms. It was used as a benchmarking AI. All Eliminator code is in `src/Eliminator.hs` beyond line 31. It has still been styled properly and documented.

Attempt 1 – Greedy

To begin, I made a simple greedy ai and a simple heuristic. This would act as a benchmark ai.

The heuristic returned the score of the current player, a modified version of the function would take a Boolean determining if it was the maximising player's turn. It would then output the correct heuristic.

For game overs, my code would output a value at the 64-bit integer limit. `(2 ^ 63)` for the upper bound and `(2 ^ 63) + 1` for the lower bound. This value is stored as `inf :: Int`.

The greedy algorithm would find the gamestate with the highest heuristic, and then select that as its next move.

Attempt 2 – Elimination of certain branches

Development:

This attempt to make an algorithm came with two goals: to look ahead and record the outcome of later gamestates; to do this efficiently by eliminating certain branches.

I defined a `Tree a` data type, along with a Heuristic tree, `HTree`, and a Gamestate tree, `GTree`.

The initial algorithm would develop a game tree of depth n, it would then apply the heuristic function to every node, summing up every heuristic value and put it into the first layer of nodes.

Next, I added a long series of helper functions to the `generateTree` function, which would use mutual recursion to prevent certain nodes from being 'generated on'.

The way this node elimination algorithm would work is it would only 'generate' on child nodes with a heuristic higher than the parent node. If no nodes of this sort existed, then it would pick the maximum-heuristic node and focus-in. Due to the nature of the heuristic, it would also 'generate' on nodes with the highest score for the opponent, as it's the most likely path they'd choose.

This algorithm had its drawbacks, however.

Firstly, it assumed that the opponent was going to act greedy. This is evidenced by the fact that it worked exceptionally well against a greedy opponent, but poorly against every other form of `noLookAhead` opponent: random selection, `firstLegalMove`, weighted heuristic etc.

Secondly, it was inefficient, as it evaluated the heuristic for gamestates multiple times in order for it to eliminate certain branches.

These drawbacks meant that the algorithm was unnecessarily slow, and ineffective against smarter opponents.

Final Implementation:

Firstly, a `GTree` is generated by the `generateTree` function. This uses many helper functions all in mutual recursion. What `generateTree` does is recursively 'generates' on a node, looks at the children of a node, flags the child nodes that have a heuristic greater than their parent/or the maximum heuristic value, and then repeat the process on each flagged child node.

This function was an attempt at producing a min-max algorithm that prematurely eliminates certain branches based on whether they appeared "immediately good". Here, the clear problem with the algorithm shows face, as it was comparing the heuristics of the parent and child players, 'generating' on nodes favouring the minimizing player.

This tree would be passed onto the `makeMove` function, which would decide on the best possible path.

The lowest-level function `singleHeuristic` takes a gamestate and returns the current score for the current player. This automatically allows the heuristic to min-max the game, by turning it into a max-max problem. Because this function did not take any other arguments, the second player was always the maximizing player.

`singleHeuristic` would be applied to every element the `GTree` using the `simpleHTree` function, converting the data type into a `HTree`.

This new `HTree` would be passed through `applyGH`, which collapses the tree to a depth of 1 by adding all of the heuristics of the lower branches. It would also subtract every second layer, to make the algorithm minmax. The intention of this summation would be to give weighting to both the late-game heuristics, as well as the gamestates closer to the computer. This is assumed to help balance out future gamestates, while reducing risk for early moves – it is assumed that low starting heuristics are harder to get out of.

This new `HTree` would then be used to find the best possible move, which used the maximise function, and several list manipulation helper functions. From this, the next move is selected.

Attempt 3 – Alpha-Beta Pruning

Development:

The Alpha-Beta Pruning (ABP) method would most likely fix the assumptive problems with the previous algorithm.

ABP, however is very technically complex, and I required further research in order to understand how it worked. Furthermore, independently developing an ABP algorithm is incredibly difficult, so I chose to reverse-engineer a similar algorithm from pseudocode[1] [2]. This still required understanding for what ABP is as a process, and also required developing Haskell code from scratch.

The pseudocode source is listed as a footnote, but it is also found in the file `src/Pruning.hs`.

This function initially did not work well, but that is most likely due to errors in the reverse-engineer process. It also is due to the lack of complexity in my heuristic method.

The initial reason why it didn't work was that it was performing the Min/Max analysis for the wrong opponent, instead selecting the worst possible option. Changing just one Boolean within the initial call of `headABPrune` was enough to beat every other algorithm.

Final Implementation:

The function `simpleHeuristic` acts very similarly to the function singleHeuristic, but it takes a Boolean that determines the maximising player. This means that the ABP algorithm does the min-maxing.

In order to ensure future modularity, I created a data type Heuristic, which allows the ABP algorithm to take in any sort of heuristic function.

The heuristic was used by the `abPrune` algorithm, which uses mutual recursion to perform an ABP by passing the values of alpha, beta, and an accumulator v. The `maxPrune`/`minPrune` functions act to recursively go through every child node and recursively call the `abPrune` function for each of them. These functions aim to update the accumulator value. The `maxAlpha`/`minBeta` functions update the values of alpha and beta, given the child node, and then to pass the tail back to the `maxPrune`/`minPrune` functions. The `expandNode` function expands on the previous node, initiating the process of child node searches

The `abPrune` function was applied to every child node of the current gamestate and given an initial call. The max value was then selected from this.

---

[1] Wikipedia. (n.d.). *Alpha-Beta Pruning*. https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning
[2] Russell, S. J. Norvig, P. (2003). *Artificial Intelligence: A Modern Approach* (2nd ed.). Upper Saddle River, New Jersey: Prentice Hall

Attempt 4 – Alpha-Beta Pruning with Weighted Heuristics

Development:

To improve the heuristic measurement, I weighed each square of the board using a multivariable function, then I summed each score (accounting for what player was there) and multiplied by the total game score.

The multivariable function takes into account the fact that the corners, edges, centre, and middle are desirable places in the order specified. To do this, I multiplied two periodic and rotationally symmetric functions:

$$bias(x, y) = 0.5 \times \cos(2\pi r/R) \times (1 + \cos(4\pi r/R))$$

R is the 'radius' of the board, half the length of the diagonal. r is the 'distance from centre' of the token being analysed.

This bias equation was formulated due to the properties of both functions, the first cosine function made sure the corners were valued, and the second made sure there was weighting surrounding the centre. The combination of both ensured that edges were better than near-edge squares.

To confirm that this approach was more effective, I modified the pruning algorithms so that custom heuristic function type could be implemented, I then ran competitions between the AIs, totalling their scores. This new heuristic increased the average score by 16.5.

The bias function has some limitations, in that it still favours regions near the corners too much, and the edges too little. But I intentionally developed it using a continuous function, so that the AI could work on a board of any size.

Final Implementation:

The function was implemented using two functions, one would calculate the unscaled bias, and another would calculate the amplitude. The `amp` function was initially meant to multiply by the `currentScore`, but it is more efficient if it is multiplied at the end. The bias and amplitude functions would be evaluated at every tile on the board, and it then would be summed up altogether using the `heuristicBoard` function.

After this, the heuristic function would take the weighted board, and then multiply it by current score.

Testing

When developing unit tests, I made sure that they were more exhaustive than before. However, it is not ideal to rely solely on unit tests for confirming the correctness of software, primarily because there is too many possible edge cases and situations to look at – especially for higher level functions. At the lower level, mainly the heuristic development level, unit tests were easy to develop.

Heuristic unit tests were much more exhaustive in terms of edge cases, but it also took into account that there were similarities between functions – the difference between simpleHeuristic and heuristic is non-existent when handling game overs, as they are identical.

I also had to manipulate the pre-existing function `assertApproxEqual` adding an extra significance parameter. My continuous heuristic function required a lower expected significance.

Other unit tests were developed to ensure that the function was outputting an expected value, less an exhaustive test, but more a test of correctness. I intentionally didn't test top-level functions on certain edge cases, as they were not meant to be functional beyond that point.

Other testing methods were employed, particularly white-box testing, which was how I fixed the error in the ABP algorithm. All functions were white-box tested and analysed diagrammatically/step-by-step.

Benchmarking was also an important testing method, as it allowed me to compare AIs on a program level. This method, however less objective, was still useful in providing a metric for improvement.

I also competed against other people's AIs. This helped to consider the many situations in which my default AI has to handle, affecting the emphasis on what parts to focus on.

Overall Reflection

Overall, this assignment was hard, but I was still successful at creating an AI that performs well. Using Haskell is what makes the problem of AI development harder. This is not a limitation of the language, however, but my own failure to conceptually understand code. Programming the assignment was also very time-consuming, a majority of the time I spent on algorithm design.

Performing a series of attempts at improving the AI was helpful in streamlining development, it allowed me to slowly develop a conceptual understanding of code. Seeing the ABP algorithm written in pseudocode/imperative form is what really helped me understand what the AI was doing, it also made the task of developing the Haskell code easier. I do not think it would be fair to expect someone at this level to do this from scratch.

The more open-ended nature of the project made the task much harder, but also more rewarding. If I did this task again Haskell would not be used.

I made sure the code was properly styled, so that it had better readability/conformed to standards. There are some parts in which the forming may make the code harder to understand – but that's a consequence of keeping lines to under the 80-character limit. I made sure to use an excessive level of helper functions, however too much can make the flow of code harder to understand.

Many of the decisions I made were done with at least the intuition of mathematical understanding along with a clear goal for the outcome. This is clear in how I developed the weighted heuristic.

I could see clearly how my final AI operated – and why it was the most effective. When testing against other AIs, the algorithm would intentionally try to keep the opponent near the centre of the board, this made it look like it was losing at the start. It would begin to surround the opponent, aiming to get the corners and edges, putting it in an advantageous position. Eventually it would have the upper advantage due to the number of pieces it had.